The include and a_kind_of matchers let us spell out in general terms what we want: a hash containing a key of 'expense_id' and an integer value. By passing one matcher into another, we've *composed* them to create a new one that specifies just the right level of detail. We'll talk more about composable matchers in *Composing Matchers*, on page 176.

When we run our updated spec, we get another failure:

```
$ bundle exec rspec
F

Failures:

  1) Expense Tracker API records submitted expenses
     Failure/Error: parsed = JSON.parse(last_response.body)

     JSON::ParserError:
       743: unexpected token at ''

« truncated »
```

Our empty post block just sends an empty string back to the client. We need to send some JSON-formatted text instead. Add the following line to your Sinatra app:

04-acceptance-specs/04/expense_tracker/app/api.rb

```
post '/expenses' do
  JSON.generate('expense_id' => 42)
end
```

Once we're generating text that matches our expected pattern, our specs will be back to passing. Of course, this passing example isn't telling us much. We've managed to fool it by returning some canned data. This practice of hard-coding return values just to satisfy an expectation—known as *sliming the test*—allows us to flesh out the spec end-to-end and then come back later to implement the behavior properly.[9]

Querying the Data

Saving expenses is all fine and good, but it'd be nice to retrieve them. We want to allow users to fetch expenses by date, so let's post a few expenses with different dates and then request the expenses for one of those dates. We expect the app to respond with just the expenses recorded on that date.

Posting one expense after another will get really old if we have to keep repeating all that code. Let's extract that helper logic into a post_expense helper method inside the RSpec.describe block:

9. https://www.youtube.com/watch?v=PhiXo5CWjYU